

Predicting NBA Game Outcomes: A Comparison of Linear and Tree-Based Models

Anuraag Dasari

Abstract—Predicting NBA game outcomes is harder than it first looks, because home teams already win about 55 to 57 percent of the time. Any model has to beat that before it means anything. This project builds a classifier that predicts whether the home team wins, using only information available before the game starts. I used 14,041 games from the 2015-16 through 2025-26 seasons and built 42 features describing each team’s recent form over five-game and ten-game windows, their rest days, and how the two teams compare to each other. Every feature is lagged by one game so that no game can contribute to its own prediction, which is the main safeguard against data leakage. I compared logistic regression, random forest, and XGBoost using a train/test split that respects time order, and tuned each one with time-series cross-validation. All three reach about 64.3 percent accuracy and 0.70 ROC-AUC on held-out games, roughly nine points above the always-pick-home baseline. The three models finish within a quarter of a percentage point of each other, which suggests the limit here comes from the information available rather than from the choice of model.

Index Terms—sports analytics, binary classification, gradient boosting, data leakage, time-series cross-validation, feature engineering

I. INTRODUCTION

Sports prediction is a useful problem for learning machine learning because the setup is clean. The label is never ambiguous, one team wins and one team loses, and new data arrives on a schedule. Basketball in particular gives teams plenty of chances to show their quality, since an NBA game involves roughly 200 combined possessions rather than being decided by one or two events. Even so, published accuracy for NBA prediction tends to sit in a fairly narrow band around 65 to 70 percent [3], [7], [9].

Two things make this problem worth studying carefully rather than just chasing a high number.

The first is that a strong baseline already exists without any model at all. Home teams win a clear majority of NBA games, so guessing “home team wins” every single time gets about 55 percent accuracy. That is the number a model actually has to beat. Comparing against a 50 percent coin flip would make any result look much better than it is.

The second is that this problem makes it easy to cheat by accident. Box score statistics like points scored, field goal percentage, and rebounds are recorded after a game ends, and they are close to a direct readout of who won. If those columns get joined to the label without thinking about timing, the model will report accuracy above 95 percent while having learned nothing that generalizes. This is a well-documented failure mode called target leakage [16].

Because of that, I built this project around getting the methodology right. The task is to predict $y = 1$ if the home team wins, using only features that exist before tip-off. My contributions are a feature pipeline where every rolling statistic is lagged by one game so it cannot see its own outcome, a fair comparison of a linear model against two tree ensembles under time-ordered validation, and an analysis of why all three models end up in the same place.

II. RELATED WORK

Early work on game prediction used rating systems. The Elo system [1], originally built for chess, updates a hidden team strength score after every result, and it is still a solid baseline in basketball today. Harville [2] applied linear models to professional football, setting up the general approach of modeling margin of victory using team effects plus a home advantage term.

Machine learning approaches showed up later. Loeffelholz et al. [3] used neural networks on 620 NBA games and reported around 74 percent accuracy, though they used season-average statistics and a random split, which makes the out-of-sample claim weaker than it first appears. Miljković et al. [4] got similar numbers with naive Bayes. Manner [7] modeled NBA outcomes with state-space models that let team strength drift over the season, and found that letting strength change over time worked better than treating it as fixed. That result is the direct reason I use rolling windows instead of season averages.

A second group of papers models games at the possession level. Štrumbelj and Vračar [5] simulated games with a Markov model over possession outcomes, and Vračar et al. [8] later modeled play-by-play sequences directly. These perform well but need detailed event data that does not exist before a game is played, so they are solving a different problem than mine.

A third group tests models against betting markets, which is a much harder bar. Hubáček et al. [9] found that beating closing lines requires information beyond aggregated team statistics. Lopez and Matthews [6] similarly showed that combining efficiency metrics with market prices beat either one alone in college basketball. Entine and Small [10] looked specifically at rest as a source of home advantage, which is why I included rest features.

On the evaluation side, Tashman [17] argues that forecast evaluation has to respect time ordering, which is the basis for my split strategy. Bradley [18] makes the case for ROC-AUC

as a threshold-independent way to compare models. Niculescu-Mizil and Caruana [19] document that boosted and bagged models often produce poorly calibrated probabilities, which is why I report log-loss alongside accuracy.

III. METHODOLOGY

A. Problem Setup

I treat this as binary classification. For game i with home team h and away team a , the label is

$$y_i = \begin{cases} 1, & \text{if the home team wins} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Every NBA game has exactly one home team and one away team, so this covers every possible result. Predicting $y_i = 0$ is the same thing as predicting an away team win. The model estimates $P(y_i = 1 \mid \mathbf{x}_i)$ where $\mathbf{x}_i \in \mathbb{R}^{42}$ holds only pre-game information.

B. Building Features Without Leakage

The one rule everything else follows is that $\mathbf{x}_i$ has to be knowable before game i starts. Let $\mathcal{G}_t = (g_1, g_2, \dots, g_{n_t})$ be the games team t has played in date order, and let $s(g)$ be a box score statistic from game g . The w -game rolling feature for team t going into its k -th game is

$$\phi_w^s(t, k) = \frac{1}{\min(w, k-1)} \sum_{j=\max(1, k-w)}^{k-1} s(g_j). \quad (2)$$

The important detail is that the sum stops at $k-1$. Game k never contributes to its own features. In code this is a one-game shift applied before the rolling average, computed separately within each team.

I computed ϕ_w^s for $w \in \{5, 10\}$ across nine statistics: points scored, points allowed, field goal percentage, three-point percentage, rebounds, assists, turnovers, a win indicator, and point differential. Doing that for both teams gives $9 \times 2 \times 2 = 36$ features. I added rest days for each side, measured as the gap since that team last played, plus two features that compare the two teams directly:

$$\Delta_{\text{form}} = \phi_{10}^{\text{ptdiff}}(h) - \phi_{10}^{\text{ptdiff}}(a), \quad (3)$$

$$\Delta_{\text{rest}} = \text{rest}(h) - \text{rest}(a). \quad (4)$$

That comes to 42 features. I kept both window sizes instead of picking one because they capture different things. The five-game window reacts quickly when a team gets hot or cold, but it is noisy since five games is a small sample. The ten-game window is steadier but slower to notice a real change. Giving the model both lets it decide how to weigh them.

C. Models

I compare three classifiers that get progressively more flexible, so that any difference between them says something about whether the extra flexibility actually helps.

Logistic regression [11] models the log-odds as a linear function of the features,

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (5)$$

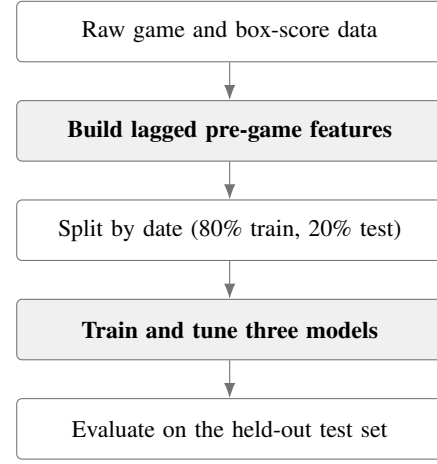


Fig. 1. Pipeline overview. The two bold steps are where leakage is prevented: features are lagged so no game sees its own result, and the split is by date rather than random.

fit by minimizing log-loss with ℓ_2 regularization controlled by C . This is the simple, interpretable floor that the other two have to beat.

Random forest [12] averages B decision trees, each grown on a bootstrap sample with a random subset of features considered at every split, so $\hat{p}(\mathbf{x}) = B^{-1} \sum_{b=1}^B T_b(\mathbf{x})$. Averaging many trees cuts variance and lets it pick up interactions a linear model cannot express.

XGBoost [14] builds trees one at a time, with each new tree correcting the errors of the ones before it, under the objective

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{v}\|^2, \quad (6)$$

where T is the number of leaves and $\mathbf{v}$ the leaf weights. It follows the gradient boosting framework from Friedman [13] and is usually the strongest option on tabular data like this.

D. Validation

Regular k -fold cross-validation does not work here. It would let the model train on games that happen after the ones it is being validated on, which is a quiet form of leakage that makes scores look better than they are [17]. So I held out the most recent 20 percent of games as a test set, and tune hyperparameters using `TimeSeriesSplit` with five expanding folds, where each fold trains on the earliest games and validates on the block right after. Every search runs inside the training portion only, and the test set is evaluated once, after tuning is finished. Fig. 1 shows the whole process.

IV. DATASET AND PREPROCESSING

A. Source

I used a public NBA dataset with two tables joined on game ID. The first (`Games.csv`) has one row per game covering 73,279 games from 1946 through June 2026, with dates, teams, scores, and the winner. The second (`TeamStatistics.csv`) has two rows per game, one for

each team, with the full box score and a home/away flag. Scores and winners have no missing values.

B. Filtering

I kept games on or after October 1, 2015. The NBA season runs across two calendar years and there are no games between July and September, so this cutoff lands inside that gap and picks up the complete 2015-16 through 2025-26 seasons without cutting any season in half. Staying in the modern era also keeps playing style roughly consistent, which matters because three-point volume changed a lot in the years before this. I kept only regular season and playoff games, dropping preseason and All-Star games since those are not played competitively.

That leaves 14,058 games. A few rows are missing box score fields and get dropped. Equation 2 is undefined for a team’s very first game, so 17 games with no history behind them are removed as well. The final dataset is **14,041 games** with 42 features, running from October 29, 2015 to June 13, 2026.

C. Data Splits

The 80/20 chronological split gives **11,232 training games** (2015-10-29 to 2024-04-02) and **2,809 test games** (2024-04-02 to 2026-06-13), so the test set covers roughly two full seasons the model has never seen. Inside the training portion, five expanding folds are used for tuning. Features are standardized for logistic regression using statistics computed on the training data only. The tree models use raw features, since scaling does not affect them.

D. Exploratory Data Analysis

Fig. 2 plots the home win rate by season, which was the main thing I checked before modeling. Overall it is 0.566, but it is not steady over time. Home advantage dropped from 0.596 in 2015-16 to around 0.545 in recent seasons, with the lowest point in 2019-20 and 2020-21, the seasons largely played without fans. This is a good reason to evaluate chronologically, since a model trained on older seasons carries a stronger home bias than the test years actually support.

I also checked how strongly each feature correlates with the label. The values are all modest, with the ten-game form edge Δ_{form} the strongest at $r = 0.295$, followed by home and away ten-game win rate at 0.204 and -0.173 . Rest advantage barely correlates at all (-0.008). No single feature carries the prediction on its own, which is an early sign that the accuracy ceiling will be modest.

V. EXPERIMENTS, RESULTS, AND DISCUSSION

A. Setup

Everything runs in scikit-learn 1.8 [15] and XGBoost 3.3 with a fixed seed of 42. I tuned with exhaustive grid search over the ranges in Table I, scoring on mean ROC-AUC across the five expanding folds. Grid search is fine here rather than random search [20] because the grids are small.

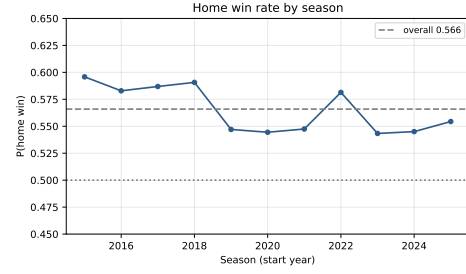


Fig. 2. Home win rate by season. Home advantage has fallen over the last decade, from about 0.596 to 0.545, with the lowest point in the 2019-20 and 2020-21 seasons.

TABLE I
HYPERPARAMETER GRIDS AND WHAT WAS SELECTED.

Model	Grid searched	Selected
Logistic Reg.	$C \in \{0.01, 0.1, 1, 10\}$	$C = 0.01$
Random Forest	trees $\in \{200, 400\}$	400 trees
	depth $\in \{5, 8, 12\}$	depth 5
XGBoost	trees $\in \{200, 400\}$	200 trees
	depth $\in \{3, 5\}$	depth 3
	$\eta \in \{0.03, 0.1\}$	$\eta = 0.03$

One pattern stands out right away. Every model picked the most conservative option available to it: the strongest regularization for logistic regression, the shallowest trees for both ensembles, and the smallest learning rate for XGBoost. That is a consistent signal that this problem only supports a fairly simple decision boundary, and that giving the models more capacity leads to overfitting rather than better predictions.

B. Metrics

I report three metrics because each one answers a different question.

Accuracy is the share of correct predictions at a 0.5 threshold. It is the number that compares directly to the always-home baseline, but it ignores how confident the model was.

ROC-AUC [18] is the probability that a randomly picked home win gets a higher score than a randomly picked home loss. It does not depend on where the threshold is set, which makes it my main metric for comparing models.

Log-loss grades the predicted probabilities rather than just their ranking:

$$\mathcal{L}_{\log} = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (7)$$

It punishes confident mistakes far harder than uncertain ones, so a model that says 0.99 and gets it wrong takes a large hit.

Precision, recall and F_1 are included for completeness. The classes here are close to balanced at 55.1 percent positive, so they matter less than they would in an imbalanced problem.

C. Training Behavior

Fig. 3 plots XGBoost training and validation log-loss against boosting iteration, using the last 15 percent of the training data

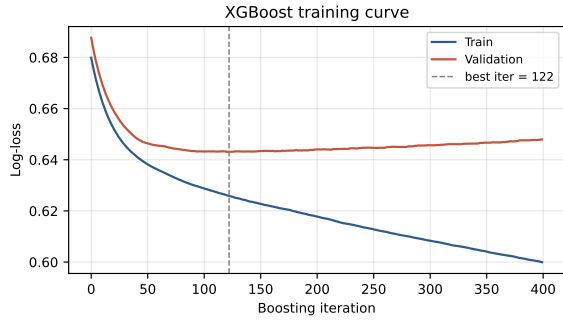


Fig. 3. XGBoost log-loss by boosting iteration. Validation loss reaches its minimum around iteration 122 and rises afterward, while training loss keeps decreasing.

TABLE II
TEST-SET RESULTS ON 2,809 HELD-OUT GAMES. BEST VALUE IN EACH COLUMN IS BOLD.

Model	Acc.	ROC-AUC	Log-loss	F_1
Always-home baseline	0.5511	0.5000	n/a	0.7106
Logistic Regression	0.6444	0.6998	0.6237	0.7049
Random Forest	0.6419	0.6986	0.6294	0.7089
XGBoost	0.6429	0.6995	0.6254	0.6965

as a validation block that still respects time order. Training loss keeps falling the whole way, but validation loss bottoms out at 0.643 around iteration 122 and then starts climbing. That gap is textbook overfitting: past that point the model is memorizing training games rather than learning anything that transfers. The tuned configuration ends up choosing 200 shallow trees, which lines up with where the curve turns.

D. Results

Table II shows how the tuned models did on the held-out test set.

Three things come out of this.

First, all three models beat the always-home baseline by 9.1 to 9.3 percentage points. That is the main result. The lagged form features carry real predictive information beyond just knowing that home teams tend to win.

Second, the numbers land in the 64 to 65 percent range that matches other published work [3], [7], [9], and nowhere near the 95 percent or higher that leakage would produce. I treat that as evidence the pipeline is correct rather than as a disappointing result.

Third, the three models are basically tied. The spread in accuracy is 0.25 percentage points and the spread in ROC-AUC is 0.0012. With 2,809 test games, the binomial standard error at $p \approx 0.64$ is about 0.9 percentage points, so those gaps sit well inside noise. There is no evidence that any of these models is truly better than the others on this data.

It is also worth pointing out that the trivial baseline gets an F_1 of 0.711, higher than two of the three real models. That happens because always predicting the majority class gives perfect recall, which inflates F_1 even though the classifier is

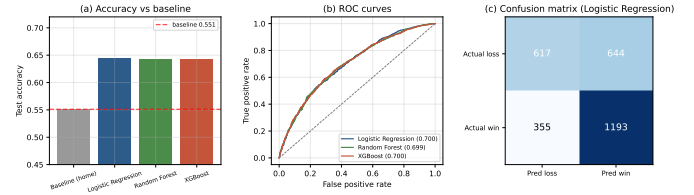


Fig. 4. Results. (a) All three models beat the baseline by about nine points. (b) ROC curves nearly overlap. (c) The confusion matrix shows a strong lean toward predicting home wins.

useless. It is a good reminder of why F_1 is the wrong headline metric for a roughly balanced problem, and why I lead with accuracy against the baseline instead.

E. Error Analysis

Fig. 4 collects the result plots. Panel (a) shows all three models clearing the baseline by a similar margin, and panel (b) shows their ROC curves sitting almost exactly on top of each other.

Panel (c) shows the confusion matrix for the best model, and it reveals a clear asymmetry. Logistic regression catches 1,193 of 1,548 home wins, a recall of 0.771, but only 617 of 1,261 home losses, a recall of 0.489. The model has absorbed the home court prior and leans toward predicting a home win whenever a game looks close. Under an accuracy objective with a 55.1 percent base rate that is the rational thing to do, but it means away team upsets get missed systematically.

F. Discussion

The main takeaway is that pre-game team-level box score data supports roughly 64 to 65 percent accuracy on NBA games, and that this ceiling comes from the data rather than from the model. Two separate pieces of evidence point the same way. Models ranging from linear to boosted ensembles all perform the same, and every hyperparameter search chose the most conservative settings on offer. If a more flexible model could pull out more signal, at least one of those would have gone the other way.

That means the remaining error is mostly about things the features cannot see. Injuries and load management change how strong a team actually is on a given night, but they do not show up in historical averages. Trades and rotation changes break the assumption that a team's last ten games describe its current roster. And basketball has a lot of irreducible randomness built in, since three-point shooting swings heavily from game to game and one hot quarter can decide a result. Getting meaningfully past 65 percent probably needs new sources of information such as injury reports, betting market prices [6], [9], or possession-level modeling [5], rather than more tuning.

Two limitations are worth stating. The test period covers seasons with a lower home win rate than the training period, as Fig. 2 shows, so a model that learned a stronger home prior is slightly penalized. That makes my reported numbers a little conservative. Also, a single split on 2,809 games has a standard error near 0.9 percentage points, so the small

differences between my three models should not be read as real differences.

VI. CONCLUSION AND FUTURE DIRECTIONS

I built and evaluated a leakage-aware pipeline for predicting NBA game outcomes from pre-game team form across 14,041 games. Using a strictly chronological split, logistic regression, random forest, and XGBoost all reached about 64.3 percent accuracy and 0.699 ROC-AUC, roughly nine percentage points above the always-home baseline. The models were statistically indistinguishable from each other, and the tuning consistently favored the most conservative settings, which together indicate that the usable signal in this feature set is low-dimensional and mostly linear.

The methodological lesson is that the hard part of this problem is the representation, not the model. Enforcing the time constraint properly, through one-game-lagged features and a chronological split, gives honest but modest accuracy. Ignoring it gives impressive numbers that mean nothing.

Future work should go after new information rather than new algorithms. The directions I would try first are adding player availability and injury report data so roster changes become visible to the model, adding a latent strength rating such as Elo [1] or a dynamic state-space rating [7] to summarize opponent-adjusted quality more efficiently than raw rolling means, benchmarking against closing betting lines to see how much inefficiency is left [9], and predicting point margin instead of a binary outcome, since collapsing a 20-point blowout and a 1-point loss into the same label throws away information that could sharpen the probability estimates.

CODE AVAILABILITY

All code, executed notebooks, and figures are available at:

https://github.com/adasari28/ML_Project

The repository has three notebooks meant to be run in order: `data_preparation.ipynb` (loading, filtering, feature engineering, and the exploratory plot), `modeling_and_tuning.ipynb` (splitting, training, the training curve, and hyperparameter search), and `evaluation_and_analysis.ipynb` (metrics and result plots). Each notebook writes files that the next one reads, so the order matters. Every figure in this report is generated by these notebooks. The README covers environment setup with instructions on the requirements, where to place `Games.csv` and `TeamStatistics.csv`, and roughly how long each notebook takes to run.

REFERENCES

- [1] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York, NY, USA: Arco Publishing, 1978.
- [2] D. Harville, "Predictions for National Football League games via linear-model methodology," *Journal of the American Statistical Association*, vol. 75, no. 371, pp. 516–524, 1980.
- [3] B. Loeffelholz, E. Bednar, and K. W. Bauer, "Predicting NBA games using neural networks," *Journal of Quantitative Analysis in Sports*, vol. 5, no. 1, 2009.
- [4] D. Miljković, L. Gajić, A. Kovačević, and Z. Konjović, "The use of data mining for basketball matches outcomes prediction," in *Proc. IEEE 8th Int. Symp. Intelligent Systems and Informatics (SIISY)*, 2010, pp. 309–312.
- [5] E. Štrumbelj and P. Vračar, "Simulating a basketball match with a homogeneous Markov model and forecasting the outcome," *International Journal of Forecasting*, vol. 28, no. 2, pp. 532–542, 2012.
- [6] M. J. Lopez and G. J. Matthews, "Building an NCAA men's basketball predictive model and quantifying its success," *Journal of Quantitative Analysis in Sports*, vol. 11, no. 1, pp. 5–12, 2015.
- [7] H. Manner, "Modeling and forecasting the outcomes of NBA basketball games," *Journal of Quantitative Analysis in Sports*, vol. 12, no. 1, pp. 31–41, 2016.
- [8] P. Vračar, E. Štrumbelj, and I. Kononenko, "Modeling basketball play-by-play data," *Expert Systems with Applications*, vol. 44, pp. 58–66, 2016.
- [9] O. Hubáček, G. Šourek, and F. Železný, "Exploiting sports-betting market using machine learning," *International Journal of Forecasting*, vol. 35, no. 2, pp. 783–796, 2019.
- [10] O. A. Entine and D. S. Small, "The role of rest in the NBA home-court advantage," *Journal of Quantitative Analysis in Sports*, vol. 4, no. 2, 2008.
- [11] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B*, vol. 20, no. 2, pp. 215–242, 1958.
- [12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [16] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [17] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *International Journal of Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.
- [18] A. P. Bradley, "The use of the area under the ROC curve in the evaluation of machine learning algorithms," *Pattern Recognition*, vol. 30, no. 7, pp. 1145–1159, 1997.
- [19] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.
- [20] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.